

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 12 实验名称 网络传输机制实验(一)

一、 实验内容

本次实验的目标是基于现有框架,实现 TCP 传输机制。主要包括以下功能:

1. 连接管理:实现服务端与客户端之间的三次握手和四次挥手。
2. 短消息传输:实现客户端向服务端发送短消息,服务端能够正确回显。
3. 大文件传输:实现客户端向服务端发送文件,服务端能够正确接收并保存文件。

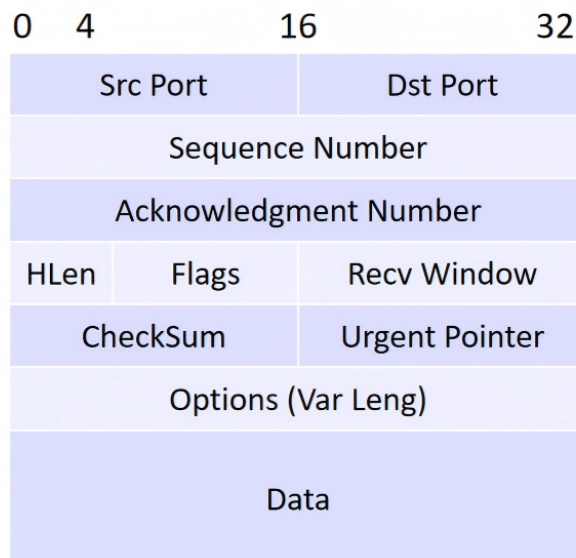


图 1: TCP 协议图

二、 实验步骤

本实验主要涉及 TCP 的连接管理、数据传输与关闭的实现步骤。

2.1 TCP 连接管理

TCP 连接管理包括三次握手和四次挥手的实现。

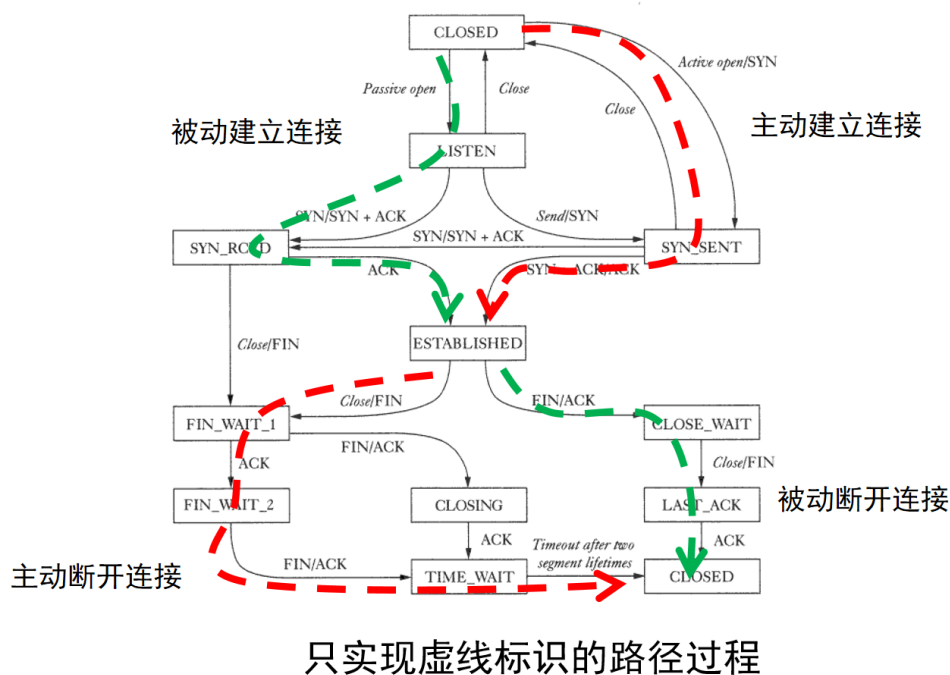


图 2: TCP 状态转移图

2.1.1 三次握手

TCP 连接的建立分为服务端(被动打开)和客户端(主动打开)两种情况:

服务端处理 服务端在 TCP_LISTEN 状态下处理连接请求:

- 收到 SYN 包:
 1. 分配一个新的子 Socket(child_tsk),并初始化其五元组信息(源/目的 IP 和端口)。
 2. 设置初始序列号(iss)和接收序列号(rcv_nxt)。
 3. 将子 Socket 加入监听队列(listen_queue)。
 4. 发送 SYN|ACK 包,并将状态转移为 TCP_SYN_RECV。
- 收到 ACK 包:
 1. 检查父 Socket 的接受队列是否已满。
 2. 将子 Socket 加入父 Socket 的全连接队列(accept_queue)。
 3. 唤醒阻塞在 accept() 上的进程。
 4. 将状态转移为 TCP_ESTABLISHED。

客户端处理 客户端在 TCP_SYN_SENT 状态下处理服务端的响应:

- 收到 SYN|ACK 包:
 1. 更新接收序列号(rcv_nxt)。
 2. 发送 ACK 包确认服务端的 SYN。
 3. 唤醒阻塞在 connect() 上的进程。
 4. 将状态转移为 TCP_ESTABLISHED。

2.1.2 四次挥手

TCP 连接的断开分为主动关闭方和被动关闭方两种情况：

主动关闭方 主动关闭方的状态转移如下：

- **TCP_FIN_WAIT_1:**
 - 如果收到 ACK 包, 状态转移为 TCP_FIN_WAIT_2。
 - 如果同时收到 FIN 包, 直接转移为 TCP_TIME_WAIT, 并设置定时器。
- **TCP_FIN_WAIT_2:**
 - 如果收到 FIN 包, 发送 ACK 包, 状态转移为 TCP_TIME_WAIT, 并设置定时器。

被动关闭方 被动关闭方的状态转移如下：

- **TCP_LAST_ACK:**
 - 如果收到 ACK 包, 状态转移为 TCP_CLOSED。

2.2 数据传输

在 TCP_ESTABLISHED 状态下, 主要处理数据接收和被动关闭请求：

- **数据接收:**
 1. 检查序列号是否有效(调用 `is_tcp_seq_valid`)。
 2. 如果数据按序到达(`cb->seq == tsk->rcv_nxt`), 将数据写入接收缓冲区, 并更新接收窗口。
 3. 发送 ACK 包确认接收到的数据。
- **被动关闭:**
 1. 如果收到 FIN 包, 更新接收序列号并发送 ACK 包。
 2. 将状态转移为 TCP_CLOSE_WAIT。

2.3 文件传输

文件传输的实现基于 TCP 协议的可靠数据传输机制。客户端通过调用 `tcp_sock_write()` 将数据发送到服务端, 服务端通过调用 `tcp_sock_read()` 接收数据并进行相应处理。

2.3.1 文件传输过程

文件传输的实现分为客户端发送文件和服务端接收文件两个部分：

客户端发送文件 客户端通过以下步骤实现文件的发送：

1. 打开待发送的文件, 并将文件内容读取到内存缓冲区中。
2. 调用 `tcp_sock_write()`, 将文件数据分块发送到服务端。每次发送的数据大小受限于 TCP 报文的最大负载长度。
3. 发送完成后, 关闭 TCP 连接。

客户端的关键代码如下：

```
FILE *fp = fopen("client-input.dat", "r");
char *data = (char *)malloc(10000000 * sizeof(char));
int data_len = 0;
while ((data[data_len++] = fgetc(fp)) != EOF);
data_len--;
fclose(fp);
tcp_sock_write(tsk, data, data_len);
free(data);
tcp_sock_close(tsk);
```

服务端接收文件 服务端通过以下步骤实现文件的接收：

1. 调用 `tcp_sock_read()`，从 TCP 接收缓冲区中读取数据。
2. 将接收到的数据写入文件，直到客户端关闭连接。
3. 关闭文件并释放资源。

三、实验结果

3.1 实验内容一：连接管理

在 h1 和 h2 上分别运行客户端和服务端程序，观察 TCP 连接的建立和断开过程。使用 Wireshark 抓包工具分析 TCP 三次握手和四次挥手的过程，结果如下：

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	b6:f3:d5:a0:5a:65	Broadcast	ARP	42	Who has 10.0.0.1? Tell 10.0.0.2
2	0.011068	9e:64:6c:d5:c6:b2	b6:f3:d5:a0:5a:65	ARP	42	10.0.0.1 is at 9e:64:6c:d5:c6:b2
3	0.021656	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [SYN] Seq=0 Win=65535 Len=0
4	0.032238	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
5	0.047189	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=1 Ack=1 Win=65535 Len=0
6	1.221506	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [FIN] Seq=1 Win=65535 Len=0
7	1.234094	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [ACK] Seq=1 Ack=2 Win=65535 Len=0
8	5.073399	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [FIN] Seq=1 Win=65535 Len=0
9	5.086903	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=2 Ack=2 Win=65535 Len=0

图 3: TCP 三次握手与四次挥手过程

```
12-tcp_stack > code > log > client_log
1  DEBUG: find the following interfaces: h2-eth0.
2  Routing table of 1 entries has been loaded.
3  DEBUG: 10.0.0.2:12345 switch state, from CLOSED to SYN_SENT.
4  DEBUG: 10.0.0.2:12345 switch state, from SYN_SENT to ESTABLISHED.
5  DEBUG: 10.0.0.2:12345 switch state, from ESTABLISHED to FIN_WAIT-1.
6  DEBUG: 10.0.0.2:12345 switch state, from FIN_WAIT-1 to FIN_WAIT-2.
7  DEBUG: 10.0.0.2:12345 switch state, from FIN_WAIT-2 to TIME_WAIT.
8  DEBUG: 10.0.0.2:12345 switch state, from TIME_WAIT to CLOSED.
9
```

图 4: client 端 TCP 连接状态变化

```
12-tcp_stack > code > log > server_log
1  DEBUG: find the following interfaces: h1-eth0.
2  Routing table of 1 entries has been loaded.
3  DEBUG: 0.0.0.0:10001 switch state, from CLOSED to LISTEN.
4  DEBUG: listen to port 10001.
5  DEBUG: 10.0.0.1:10001 switch state, from LISTEN to SYN_RECV.
6  DEBUG: 10.0.0.1:10001 switch state, from SYN_RECV to ESTABLISHED.
7  DEBUG: accept a connection.
8  DEBUG: 10.0.0.1:10001 switch state, from ESTABLISHED to CLOSE_WAIT.
9  DEBUG: 10.0.0.1:10001 switch state, from CLOSE_WAIT to LAST_ACK.
10 DEBUG: 10.0.0.1:10001 switch state, from LAST_ACK to CLOSED.
11
```

图 5: server 端 TCP 连接状态变化

把一端替换为 tcp_stack_conn.py 脚本, 依旧可以观察到 TCP 连接的建立和断开过程, 说明实现的 TCP 连接管理功能正确。

3.2 实验内容二:短消息收发

在 h1 和 h2 上分别运行客户端和服务端程序, 观察短消息的发送和接收过程。使用 Wireshark 抓包工具分析短消息的传输过程, 结果如下:

```
12-tcp_stack > code > log > ! client_log_echo
1  Routing table of 1 entries has been loaded.
2  server echoes: 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0
3  server echoes: 123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ01
4  server echoes: 23456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ012
5  server echoes: 3456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123
6  server echoes: 456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ01234
7
```

图 6: 短消息收发结果 (client 端显示)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=1 Ack=1 Win=65457 Len=0
2	0.000001	10.0.0.2	10.0.0.1	TCP	117	12345 → 10001 [PSH, ACK] Seq=1 Ack=1 Win=65457 Len=63
3	0.010470	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [ACK] Seq=1 Ack=64 Win=65472 Len=0
4	0.010479	10.0.0.1	10.0.0.2	TCP	132	10001 → 12345 [PSH, ACK] Seq=1 Ack=64 Win=65472 Len=78
5	0.030685	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=64 Ack=79 Win=65457 Len=0
6	0.030685	10.0.0.2	10.0.0.1	TCP	117	12345 → 10001 [PSH, ACK] Seq=64 Ack=79 Win=65457 Len=63
7	0.044197	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [ACK] Seq=79 Ack=127 Win=65472 Len=0
8	0.044204	10.0.0.1	10.0.0.2	TCP	132	10001 → 12345 [PSH, ACK] Seq=79 Ack=127 Win=65472 Len=78
9	0.055537	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=127 Ack=157 Win=65457 Len=0
10	0.055540	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [FIN, ACK] Seq=127 Ack=157 Win=65457 Len=0
11	0.066353	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [ACK] Seq=157 Ack=128 Win=65535 Len=0
12	0.066620	10.0.0.1	10.0.0.2	TCP	54	10001 → 12345 [FIN, ACK] Seq=157 Ack=128 Win=65535 Len=0
13	0.077037	10.0.0.2	10.0.0.1	TCP	54	12345 → 10001 [ACK] Seq=128 Ack=158 Win=65457 Len=0

图 7: wireshark 抓包分析短消息收发过程

使用 tcp_stack_trans.py 替换其中任意一端, 对端都能正确收发数据

3.3 实验内容三:大文件传送

执行 create_randfile.sh, 生成待传输数据文件 client-input.dat, 大小约为 4MB。分别在 h1 和 h2 上运行客户端和服务端程序, 观察文件传输过程。使用 md5sum 命令验证传输前后的文件一致性, 结果如下:

```
12-tcp_stack > code > ≡ client-input.dat
1  u5a8NUVVYwJfM2G5GddbroH2+ci+Jeaie+cFD20zdvx11zBwRKfn6vfvQfzH6+pLKkf0/mXlMAW0
2  GLJZvCBfrfN8NwtcFrHR+2sijdanGywd/laPQnV1LMWwJpZjJsv2NNXhE12WdIAOGkCEwr27MtdT
3  rsA36qmpR2Vz3XpT0qp0iDsm2VNwMr0i6YQghZpEsSL55vtyhQ7I2I2I9eBEZo7iuJLTm4smaZnH
4  09+hmtI+BmHk71q8/OjHQu3LxjsVSnrRexGDL+qI30V1uG6+qModn7yaCnT2I6yRXhGtbqn8ujrd7
5  5UCvEtPvBn3wDQuX8g/IEhdm2V6KmtIkzmLDkn0wTw/oS7VaJemyGxPiyb8WoEuS+RxrQaxzIB5R
6  e0NsELf7zsrwpukfxHHpMp+RXz6CGB1zNHpyXU+7jq701A494A2/VnV8yJlUk+g+D7JGa0ikpMEX
7  sRFn70P+j8i0HYtjd/6lPA85mB7oP3kPiBxrc8jHeZiYem2ARJ3/JRR9UncddUJ6flwLjd3Q4w3w
8  MFSYhphnr4biHnps9izq/OqMUNwWfiq98bzSuf07yLxCrtfzsqT8lU6iYpwUK5NzOBzIN2xRr+Z
9  8fHjx2m9MumlvAJ4lUt+EidQ04HhQ1l2BYSp/PZpThR3/q7FhcCzxsSIVJiQTjav+A4U6PwSEjUt
10 QY8Ymi+B8Z2Ugm8CpF1ZsNFxMCEqhGM+ujkPxkBugq9bdZsDk/e2DtTP5vBnfORDBpipw1QbeSPR

12-tcp_stack > code > ≡ server-output.dat
1  u5a8NUVVYwJfM2G5GddbroH2+ci+Jeaie+cFD20zdvx11zBwRKfn6vfvQfzH6+pLKkf0/mXlMAW0
2  GLJZvCBfrfN8NwtcFrHR+2sijdanGywd/laPQnV1LMWwJpZjJsv2NNXhE12WdIAOGkCEwr27MtdT
3  rsA36qmpR2Vz3XpT0qp0iDsm2VNwMr0i6YQghZpEsSL55vtyhQ7I2I2I9eBEZo7iuJLTm4smaZnH
4  09+hmtI+BmHk71q8/OjHQu3LxjsVSnrRexGDL+qI30V1uG6+qModn7yaCnT2I6yRXhGtbqn8ujrd7
5  5UCvEtPvBn3wDQuX8g/IEhdm2V6KmtIkzmLDkn0wTw/oS7VaJemyGxPiyb8WoEuS+RxrQaxzIB5R
6  e0NsELf7zsrwpukfxHHpMp+RXz6CGB1zNHpyXU+7jq701A494A2/VnV8yJlUk+g+D7JGa0ikpMEX
7  sRFn70P+j8i0HYtjd/6lPA85mB7oP3kPiBxrc8jHeZiYem2ARJ3/JRR9UncddUJ6flwLjd3Q4w3w
8  MFSYhphnr4biHnps9izq/OqMUNwWfiq98bzSuf07yLxCrtfzsqT8lU6iYpwUK5NzOBzIN2xRr+Z
9  8fHjx2m9MumlvAJ4lUt+EidQ04HhQ1l2BYSp/PZpThR3/q7FhcCzxsSIVJiQTjav+A4U6PwSEjUt
10 QY8Ymi+B8Z2Ugm8CpF1ZsNFxMCEqhGM+ujkPxkBugq9bdZsDk/e2DtTP5vBnfORDBpipw1QbeSPR

kouyixi@ubuntu:~/CNLab/12-tcp_stack/code$ md5sum client-input.dat
a175ef036a7c9f190a93ca38abbd3dc4 client-input.dat
kouyixi@ubuntu:~/CNLab/12-tcp_stack/code$ md5sum server-output.dat
a175ef036a7c9f190a93ca38abbd3dc4 server-output.dat
```

图 8: 大文件传输结果及 MD5 校验

可以看到,传输前后的文件 MD5 值一致,说明文件传输功能实现正确。使用 tcp_stack_trans.py 替换其中任意一端,对端都能正确收发数据,说明实现的 TCP 数据传输功能正确。

四、 实验中遇到的问题

在实验过程中,遇到了一些问题和挑战,主要包括以下几个方面:

1. **代码量大且复杂:** TCP 协议本身较为复杂,实现过程中需要处理多种状态和边界情况,同时本身代码框架中提供了很多辅助函数和数据结构,理解和使用这些代码需要花费较多时间。
2. **提交 oj 系统有隐形要求:**在实现过程中,发现 oj 系统对代码的某些实现细节有隐形要求,例如程序实现需要在 5s 内完成,同时 echo 需要回显在终端中,同时不能在终端打印多余信息等,这些要求在实验文档中并未明确指出,导致调试过程中出现了一些困惑。
3. **调试困难:**由于 TCP 协议涉及多个状态和复杂的交互过程,调试过程中需要仔细分析每个状态下的行为,使用 Wireshark 等工具进行抓包分析,同时需要对代码插入详细的 log 消息,以便定位问题。